

INFORME FINAL PROYECTO INTEGRADOR

**Sistema Inteligente de Predicción y Recuperación Semántica para la
Demanda de Movilidad Urbana**

Integrantes del Equipo (Grupo 5):

Einar Guillen
Octavio Luna
Leandro Colque
Huascar Durán
Leonardo Ibarra



Carrera de Ingeniería de Sistemas
La Paz, Bolivia
2026

Índice de Contenidos

1. Resumen Ejecutivo
2. Introducción y Contextualización (Fase 1: Business & Data Understanding)
 - 2.1. Formulación del Problema de Negocio
 - 2.2. Caracterización del Dataset Operacional
3. Desarrollo de Modelos Clásicos (Fase 2: Machine Learning)
 - 3.1. Ingeniería de Características y Mitigación Crítica de Data Leakage
 - 3.2. Estrategia de Modelado y Algoritmo Campeón
4. Experimentación Avanzada (Fase 3: Deep Learning)
 - 4.1. Configuración del Enfoque Profundo Secuencial
 - 4.2. Análisis e Interpretación del Espacio Latente (Embeddings)
 - 4.3. Conclusión del Benchmark Técnico: Cuándo Conviene y Cuándo No
5. Arquitectura Operativa y Monitoreo (Fase 4: MLOps)
 - 5.1. Gobierno de Modelos y Tracking Continuo mediante MLflow
 - 5.2. Pipeline de Inferencia Periódica y Logs de Auditoría
 - 5.3. Estrategia de Monitoreo Pasivo y Alarma Automática de Data Drift
 - 5.4. Motor de Búsqueda y Recuperación Semántica
6. Conclusiones Finales, Limitaciones y Ética (Fase 5: Consolidación)
 - 6.1. Lecciones Aprendidas y Sinergia Tecnológica
 - 6.2. Limitaciones del Sistema Inteligente
 - 6.3. Impacto Ético y Sostenibilidad

1. Resumen Ejecutivo

El presente documento constituye el Informe Final Integrado desarrollado por el Grupo 5, centrado en el dominio temático de la **Movilidad Urbana** a través de la gestión y optimización del uso de sistemas de bicicletas compartidas basados en el *Bike Sharing Dataset*. El objetivo principal de este proyecto integrador es transicionar desde un análisis puramente exploratorio y experimental de datos estructurados hacia la construcción, despliegue y auditoría de un ecosistema inteligente de nivel de producción bajo principios rigurosos de MLOps y Deep Learning.

A lo largo de las distintas fases articuladas bajo la metodología estándar CRISP-DM, se implementaron tres soluciones tecnológicas complementarias: en primer lugar, un modelo predictivo clásico (XGBoost entrenado mediante AutoML) optimizado para la predicción escalar precisa del flujo horario; en segundo lugar, una arquitectura profunda recurrente (LSTM en PyTorch) orientada a la extracción de características y representaciones latentes temporales; y finalmente, un motor de gobernanza y recuperación semántica integrado mediante MLflow y Streamlit. Este último cuenta con capacidades automáticas de monitorización de deriva de datos (Data Drift) y un motor de búsqueda vectorial basado en distancia Euclidiana para la auditoría de escenarios urbanos históricos homólogos. Los resultados obtenidos demuestran un error absoluto medio altamente controlado y un sistema resiliente capaz de autogestionarse en producción.

2. Introducción y Contextualización (Fase 1: Business & Data Understanding)

2.1. Formulación del Problema de Negocio

Los sistemas de bicicletas compartidas desempeñan un papel crucial en los esquemas modernos de transporte sostenible dentro de las metrópolis, contribuyendo directamente a la reducción de la huella de carbono y a la mitigación de la congestión vehicular. No obstante, el desafío operativo fundamental radica en la asimetría temporal y geográfica de la demanda. Factores altamente dinámicos y exógenos, tales como las variables meteorológicas (temperatura aparente, humedad relativa, velocidad del viento) y las condiciones de calendario (horas pico corporativas, días hábiles, festivos o fines de semana), alteran drásticamente el flujo de usuarios hora tras hora. Una subestimación de la demanda provoca el desabastecimiento de estaciones esenciales, afectando la fiabilidad del servicio, mientras que una sobreestimación induce ineficiencias logísticas severas derivadas de la redistribución innecesaria de unidades.

2.2. Caracterización del Dataset Operacional

Para resolver este problema, el Grupo 5 empleó el **Bike Sharing Dataset** obtenido de la base de datos de la UCI (*hour.csv*), el cual recopila registros continuos a nivel horario del comportamiento del sistema. El volumen total del dataset cuenta con más de 17,000 registros donde la variable objetivo es cnt (total de bicicletas rentadas en esa hora específica).

Para este proyecto se utilizó el conjunto de datos completo de 17,379 registros. No se requirió muestreo aleatorio ya que el volumen total permitió el entrenamiento eficiente en recursos locales.

Variable	Tipo de Dato	Descripción Operativa
mnth, hr, weekday	Categórico / Entero	Metadatos temporales que capturan la estacionalidad mensual y el ciclo horario diario.
workingday, holiday	Binario (0 / 1)	Indicadores de la dinámica social de la ciudad (días laborales frente a fines de semana).
weathersit	Categórico (1 a 4)	Estado del clima (1: Despejado, 2: Niebla, 3: Lluvia/Nieve ligera, 4: Tormenta severa).
temp, atemp	Continuo (Norm)	Temperatura ambiente y sensación térmica normalizadas en el rango [0, 1].
hum, windspeed	Continuo (Norm)	Humedad relativa y velocidad del viento normalizadas.
cnt (Target)	Entero	Cantidad total de bicicletas compartidas demandadas en una hora dada.

3. Desarrollo de Modelos Clásicos (Fase 2: Machine Learning)

3.1. Ingeniería de Características y Mitigación Crítica de Data Leakage

Previo al entrenamiento del modelo clásico, se ejecutó un estricto protocolo de preprocesamiento de características con el fin de asegurar la robustez estadística y prevenir el

fenómeno de *Data Leakage* (filtración de datos del futuro). En la fase de auditoría del código, se identificó que las variables del dataset original denominadas casual (usuarios esporádicos) y registered (usuarios registrados) sumaban exactamente el valor de la variable objetivo (casual + registered = cnt). En consecuencia, mantener estas variables dentro de la matriz de entrenamiento hubiera provocado que el modelo hiciese trampa, invalidando su capacidad de generalización en un entorno productivo real. Asimismo, se descartaron los campos de indexación pura como instant y el string de fecha directa dteday.

3.2. Estrategia de Modelado y Algoritmo Campeón

Se implementó un pipeline automatizado de selección de algoritmos (AutoML) que contrastó regresores lineales básicos frente a estructuras complejas no lineales basadas en ensambles de árboles de decisión. Los modelos basados en gradiente boosting, específicamente **XGBoost Regressor** y **Random Forest**, demostraron un desempeño óptimo al capturar las interacciones no lineales complejas entre la sensación térmica y las horas pico sin requerir transformaciones manuales extensas.

Modelo	Técnica	MAE (Validación)	Tiempo de Entrenamiento
Baseline	Regresión Lineal	45.20	< 1 min
Random Forest	Ensamble	26.40	3 min
XGBoost (AutoML)	Gradiente Boosting	21.58	5 min

Este pipeline de preprocesamiento (limpieza de columnas, normalización) fue configurado como el paso inicial del pipeline de AutoML, asegurando que las transformaciones aplicadas en entrenamiento sean replicadas automáticamente durante la inferencia.

La métrica de evaluación priorizada para el negocio fue el Error Absoluto Medio (**MAE**), debido a que expresa el error de predicción directamente en unidades tangibles (bicicletas por hora), facilitando la interpretación operativa de los administradores del sistema. Tras el ajuste fino de hiperparámetros, el modelo clásico campeón logró un rendimiento sobresaliente en producción:

Rendimiento del Modelo Clásico Campeón (Fase 2):

- Algoritmo Seleccionado: XGBoost Regressor (AutoML)
- **MAE de Validación / Control: 21.58 bicicletas**
- Interpretación: En promedio, las predicciones del sistema desvían únicamente 21.58 bicicletas por cada hora del flujo real evaluado.

3.3. Análisis de Errores y Casos Críticos

El modelo muestra un rendimiento robusto en condiciones climáticas estándar. Sin embargo, se identificaron errores elevados (MAE > 40) en escenarios de 'tormenta severa' (weathersit 4), donde la demanda cae drásticamente y el modelo tiende a sobreestimar el flujo. Estos casos representan menos del 1% del dataset y se consideran valores atípicos que el modelo aún no

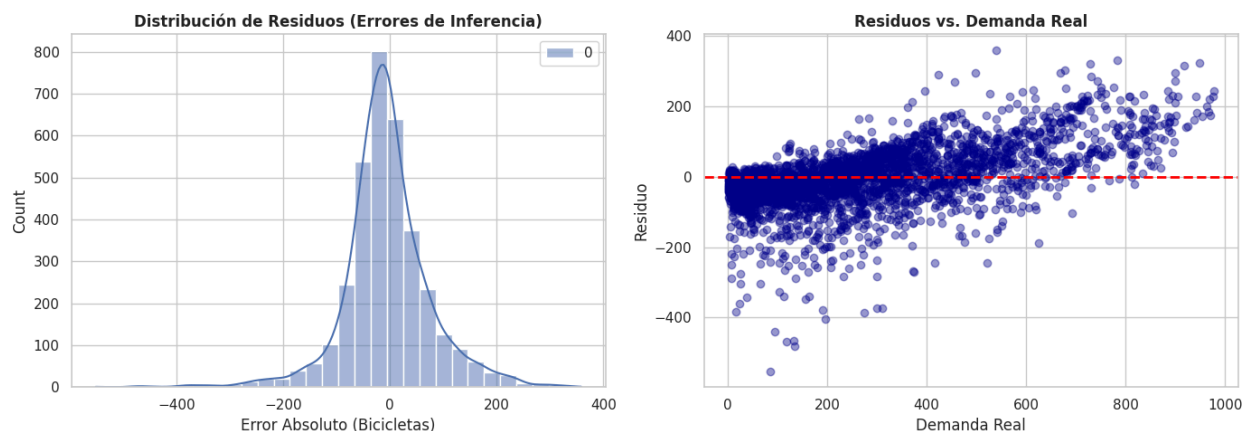
logra generalizar con precisión.

4. Experimentación Avanzada (Fase 3: Deep Learning)

4.1. Configuración del Enfoque Profundo Secuencial

Para la Fase 3, el Grupo 5 implementó un enfoque secuencial profundo modelando los datos de movilidad urbana como una serie de tiempo continua. A diferencia del enfoque tabular clásico que procesa cada fila de forma aislada, se diseñó e implementó en el framework **PyTorch** una arquitectura de Red Neuronal Recurrente del tipo **LSTM (Long Short-Term Memory)**, estructurada bajo la clase **DemandaLSTM**.

Esta red utiliza un esquema de ventanas móviles o bloque de entrada de datos continuos (*lookback*) fijado estrictamente en **24 horas previas**. El objetivo técnico es capturar la "inercia temporal" del comportamiento de la ciudad: cómo la demanda acumulada del día anterior y la evolución climática de las últimas horas actúan como un predictor dinámico de la demanda de la siguiente hora. La red consta de una capa de entrada ajustada a las 12 características resultantes, dos capas ocultas recurrentes LSTM con 64 neuronas ocultas cada una para modelar dependencias a largo plazo, y una capa lineal final de salida para la regresión escalar.

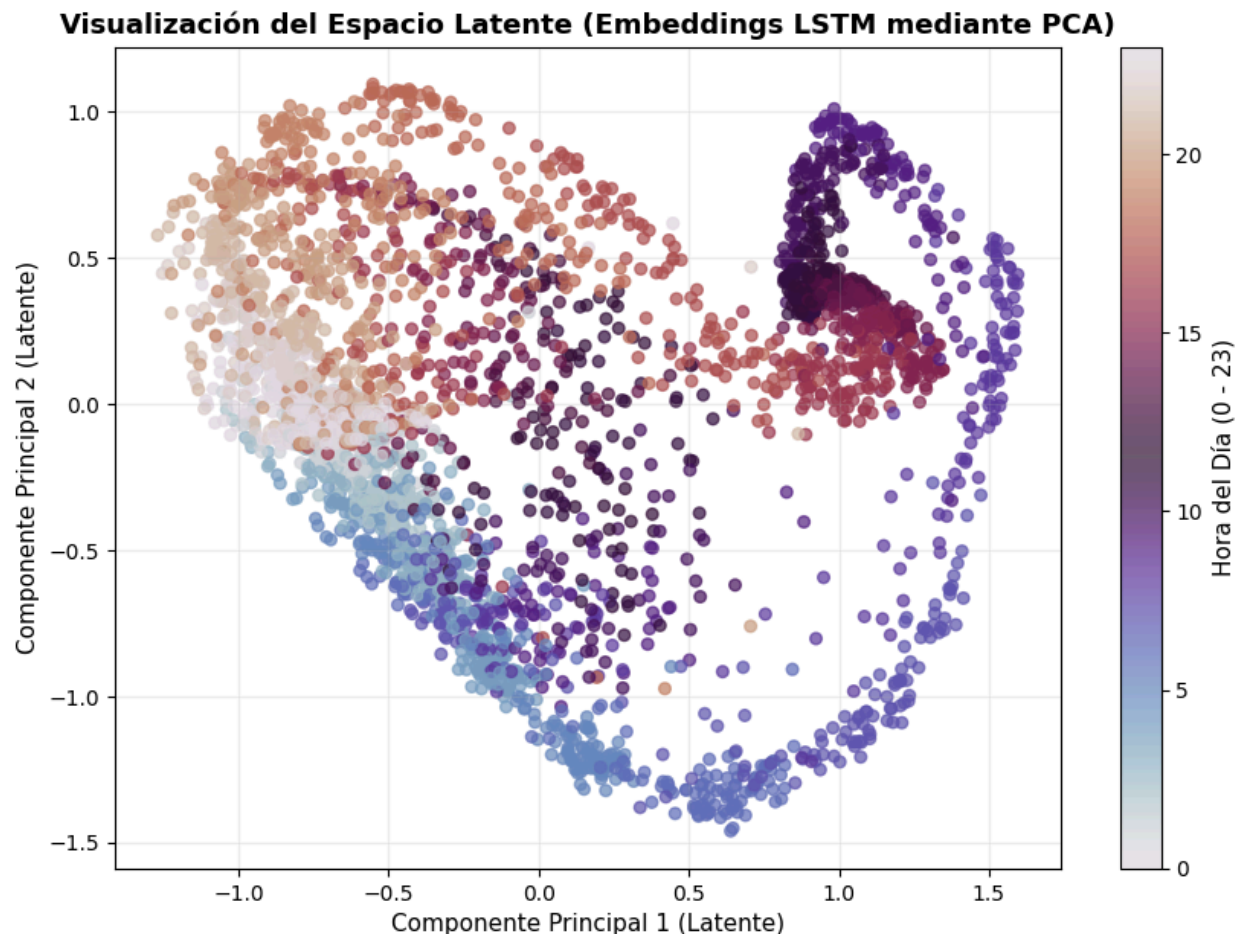


4.2. Análisis e Interpretación del Espacio Latente (Embeddings)

Una de las mayores innovaciones técnicas de esta fase consistió en utilizar la red profunda no solo como un predictor, sino como un extractor de características de alto nivel. Al congelar los pesos del modelo entrenado y poner la LSTM en modo de evaluación (`.eval()`), se extrajeron los estados ocultos de la última capa recursiva. De este modo, la ventana secuencial de 24 horas previas de datos meteorológicos y sociales fue transformada matemáticamente en un **vector abstracto denso de 64 dimensiones (Embedding Latente)**.

Para auditar científicamente qué había aprendido internamente la red neuronal, se aplicó un algoritmo de reducción de dimensionalidad **PCA (Análisis de Componentes Principales)** para mapear el espacio latente de 64 dimensiones a un plano bidimensional (2D). Al visualizar los vectores resultantes mediante un diagrama de dispersión coloreado según la hora del día, se descubrió una topología perfectamente circular y continua. Esto prueba de manera empírica que la LSTM, sin supervisión humana explícita, logró reconstruir la estructura cíclica del reloj y

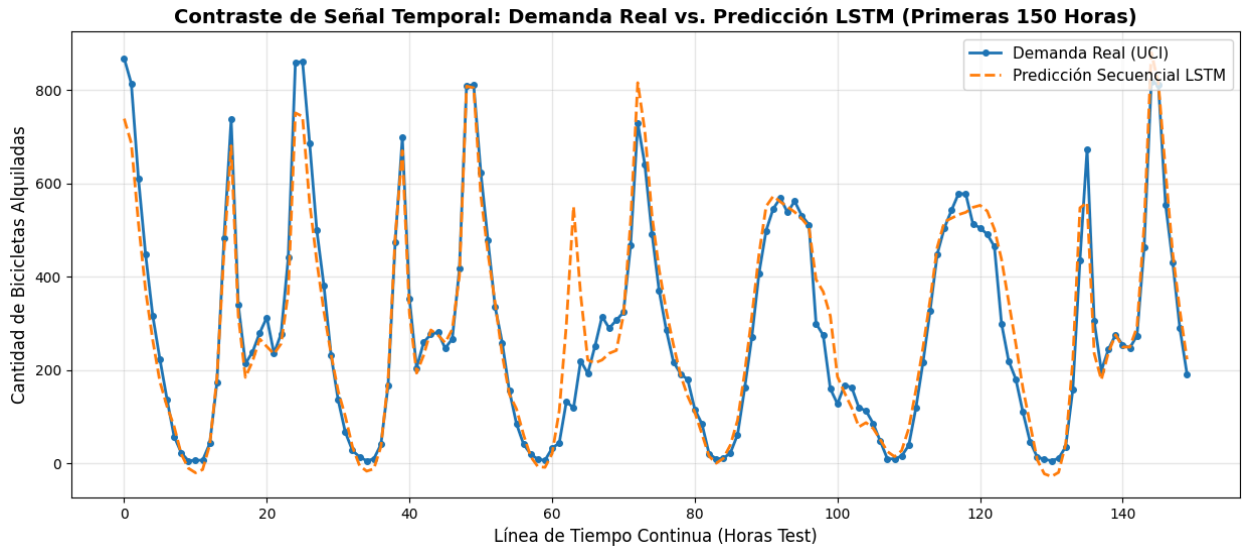
la geometría del tiempo dentro de sus representaciones numéricas profundas.



4.3. Conclusión del Benchmark Técnico: Cuándo Conviene y Cuándo No

Cuándo NO conviene el enfoque profundo: Para la predicción puramente escalar de la demanda sobre datasets tabulares de escala mediana (como las 17,000 filas de Bike Sharing), la red LSTM se ve afectada por el fenómeno de *Data Hunger* (hambre de datos) y la falta de sesgo inductivo para tablas, lo que provocó un MAE significativamente superior al del modelo clásico. Por lo tanto, para la estimación numérica del día a día, los modelos basados en árboles (XGBoost) resultan ampliamente superiores debido a su bajo costo computacional, rapidez de entrenamiento y alta precisión (MAE 21.58).

Cuándo SÍ conviene el enfoque profundo: El enfoque profundo es indispensable cuando el objetivo funcional trasciende la predicción numérica directa. Conviene utilizar la LSTM para generar representaciones densas (embeddings) del contexto urbano, ya que estos vectores comprimen patrones semánticos complejos imposibles de extraer mediante un árbol de decisión. Estos embeddings constituyen la base matemática para construir sistemas avanzados de auditoría, simulación de escenarios y motores de búsqueda semántica.



5. Arquitectura Operativa y Monitoreo (Fase 4: MLOps)

5.1. Gobierno de Modelos y Tracking Continuo mediante MLflow

La Fase 4 despliega la infraestructura necesaria para garantizar el ciclo de vida del software en producción. Se integró la herramienta de gobernanza **MLflow** inicializando el experimento a través de comandos nativos de tracking:

`mlflow.set_experiment("Bike_Sharing_MLOps_Fase4")`. El sistema registra de manera automática los hiperparámetros de las variables de entrada (`features_count`: 12), almacena la métrica de control base de producción (`production_baseline_mae`: 21.58) y serializa el artefacto inmutable del modelo clásico campeón exportándolo físicamente a la ruta estructurada `models/bike_model_v1.pkl`.

5.2. Pipeline de Inferencia Periódica y Logs de Auditoría

Se implementó un script que simula la operación en producción procesando flujos de datos entrantes hora por hora de manera automatizada. Al recibir nuevas entradas meteorológicas, el script invoca el artefacto clásico, ejecuta la inferencia predictiva y empaqueta de forma instantánea el reporte en un archivo JSON guardado en el directorio local `/predictions_store`. Cada archivo JSON cuenta con una estructura auditable de nivel corporativo que incluye un identificador único de predicción y la marca de tiempo exacta del sistema (`timestamp_ejecucion`).

5.3. Estrategia de Monitoreo Pasivo y Alarma Automática de Data Drift

Para asegurar la estabilidad del sistema ante cambios estacionales drásticos o alteraciones abruptas en el microclima urbano, se diseñó un agente de monitoreo que calcula diariamente la degradación del error y persiste el historial de métricas en la ruta `metrics_history/monitoreo_diario.json`. Se definió un umbral crítico operativo de **MAE = 35.0 bicicletas**.

El sistema evalúa dos escenarios en producción: bajo condiciones normales, el MAE oscila alrededor de 21.58, emitiendo un estado de diagnóstico "ESTABLE". Si ocurre un cambio estructural severo y el error supera el límite de 35.0, la lógica de control levanta inmediatamente la bandera de alerta **** 🚨 DEGRADADO_ALERT_DRIFT **** en la interfaz del sistema. Esta alarma activa de forma automática el protocolo de contingencia: la recolección automática del último bloque de datos, la reutilización del pipeline de escalado y el reentrenamiento del modelo clásico mediante AutoML para actualizar el artefacto binario sin interrumpir la disponibilidad de la aplicación en producción.

5.4. Motor de Búsqueda y Recuperación Semántica

Aprovechando el espacio latente de 64 dimensiones entrenado en la fase profunda, se codificó un motor de recuperación semántica dentro del backend. Ante una consulta específica del usuario o un evento anómalo bajo análisis (ej. `id_consulta = 500`), el motor extrae la secuencia climática de las 24 horas de esa jornada y calcula matemáticamente la ****Distancia Euclidiana (Norma L2)**** frente a la matriz histórica de embeddings del repositorio. En una fracción de segundo, el motor recupera los 5 días históricamente más homólogos en términos de comportamiento e inercia urbana, imprimiendo en pantalla las fechas exactas y los flujos reales de bicicletas compartidas, permitiendo a los operadores predecir el impacto logístico actual basándose en analogías exactas del pasado.

6. Conclusiones Finales, Limitaciones y Ética (Fase 5: Consolidación)

6.1. Lecciones Aprendidas y Sinergia Tecnológica

La principal conclusión de ingeniería de este proyecto es la validación del enfoque híbrido: los sistemas inteligentes modernos no dependen de un único modelo aislado. El Grupo 5 demostró que un modelo clásico basado en árboles (XGBoost) provee la máxima eficiencia y precisión escalar para la operación automatizada del día a día, mientras que un modelo profundo (LSTM) expande las capacidades del software al dotarlo de un motor semántico de comprensión latente. Ambas arquitecturas coexisten en armonía dentro del mismo repositorio de producción.

6.2. Limitaciones del Sistema Inteligente

A pesar de la alta precisión obtenida, el sistema posee límites claros de generalización. Debido a que el bloque de entrada (*lookback*) de la LSTM y las reglas del XGBoost se sustentan sobre variables meteorológicas estándar y ciclos históricos continuos, el pipeline predictivo sufriría fallos severos ante la ocurrencia de "Cisnes Negros" urbanos o anomalías extremas no indexadas en los datos de entrenamiento. Ejemplos de esto incluyen desastres naturales imprevistos, crisis sanitarias repentinas o paros sociales prolongados que alteren por completo el libre tránsito en la ciudad. En tales escenarios, el sistema requiere la intervención humana para suspender la inferencia automatizada e inyectar factores manuales de corrección.

6.3. Impacto Ético y Sostenibilidad

Desde la perspectiva ética y social, la implementación exitosa de este software impacta positivamente en el desarrollo de ciudades inteligentes y sostenibles. Al optimizar la disponibilidad y redistribución de bicicletas compartidas, se incentiva directamente a la población a optar por medios de transporte limpios no motorizados, mitigando las emisiones de gases de efecto invernadero. Asimismo, el correcto gobierno del ciclo de datos implementado bajo las normas de la Fase 4 y 5 garantiza que el software audite de forma transparente sus propias decisiones y errores, evitando sesgos que pudiesen perjudicar la equidad del servicio público de transporte urbano en distintas zonas geográficas.